sotX v0.2.0

A Verified Microkernel BSD Personality with
Deception, SMF, Project Crossbow and FMA

Architecture, Implementation, and Proposal

Lucas Sotomayor
lucas@base4.io
BASE4 Security

2026-04-08
Codename: Project STYX

Abstract

sotX is a verified microkernel operating system written from scratch in Rust, layered with a SOT (Secure Object) exokernel, a NetBSD `rump` kernel personality, a Linux ABI personality, a deception subsystem with capability interposition and live migration, and a v0.2.0+ "Illumos Heirlooms" wave that adds a Solaris-style Service Management Facility (SMF), Project Crossbow virtual networking, and a Fault Management Architecture (FMA) predictive self-healing engine. The codebase is 108k+ LOC of Rust across 69 crates, 4.4k LOC of necessary C glue (Limine + LKL/rump shims), 6 TLA+ specifications under TLC model checking, and 16 PASS markers verified end-to-end in a single QEMU boot. This document describes the architecture, the threat model the system addresses, the v0.2.0 deliverables, and the technical proposal for what sotX enables that no other open-source operating system in 2026 does.

Contents

1 Executive Summary **2**

2 Threat Model and Motivation **3**
2.1 What sotX defends against . . . 3
2.2 Why a microkernel for this . . . 4

3 Architecture **4**
3.1 The five-primitive microkernel . . . 4
3.2 The SOT exokernel layer . . . 4
3.3 KARL: per-boot ID randomisation . . . 5

4 The v0.2.0 Illumos Heirlooms Wave **5**
4.1 SMF (Service Management Facility) . . . 5
4.2 Project Crossbow: virtual networking . . . 6
4.3 FMA (Fault Management Architecture) . . . 7

5 The Wayland Compositor Renewal **8**
5.1 Visual upgrade summary . . . 8
5.2 Compositor architecture invariants . . . 8

6 Boot Verification: 16 PASS Markers **8**

7 Production Hardening **9**
7.1 Strict CI gate . . . 9
7.2 Audit + supply chain . . . 9
7.3 Reproducible builds . . . 9

7.4 ABI fuzzing and ABI diff bot 9
7.5 Reproducible release pipeline 10

8 Formal Verification **10**

9 Codebase Inventory **10**

10 What sotX enables **10**

11 Roadmap and known follow-ups **11**
11.1 Post-v0.2.0 follow-ups (non-blocking) 11
11.2 Planned for v0.3.0 12

12 Conclusion **12**

1 Executive Summary

sotX is the BSD personality branch of `sotX`, a research microkernel built from scratch over the past year. As of v0.2.0 (commit `eb137e3`, 2026-04-08) it ships:

- A 5-primitive verified microkernel (scheduling, IPC, capabilities, IRQ virtualisation, frame allocation), ∼10 k LOC, with capability tables, CDT, per-core ticket-locked run queues, work stealing, EDF deadline scheduling, multi-address-space processes, and SMP on 1–4+ cores.
- A SOT (Secure Object) exokernel layer with 12 syscalls (300–311) for transactional secure objects, single-tier and two-phase commit, capability interposition with proxy domains, and a per-CPU 4096-entry provenance ring.
- Real **NetBSD rump kernel** linked into a freestanding sotX service: `librump.a + librumpvfs.a + librumpfs_ffs.a + librumpdev.a + librumpdev_disk.a + ...` (∼2 MiB) bound through 60 hand-rolled `rumpuser_*` hypercalls in `rumpuser_sot.c`.
- A **Linux ABI personality** (LUCAS) with full `fork/execve/waitpid`, glibc and musl static + dynamic binaries, vDSO trampolines, full `mmap/brk`, signal delivery (sync + async via timer ISR), real `clone(CLONE_THREAD)`, futex, TLS, and the `services/lucas-shell` command interpreter.
- A **deception subsystem** (`personality/common/ deception`) with anomaly detection (*credential theft after backdoor, reconnaissance sweep, data exfiltration, persistence install, staged payload, privilege escalation, anomalous fan-out*), Migration Orchestrator, and four deception profiles (Ubuntu webserver, CentOS database, IoT camera, Windows file server) wired through capability interposition.
- An **Ed25519 + SHA-256 boot chain**: every signed binary in the initrd is hashed and Ed25519-verified against a public key compiled into init via `services/init/src/sigkey_generated.rs`. Pubkey-pinning rejects swap-with-self-signed attempts.
- The **v0.2.0+ Illumos Heirlooms wave**: SMF (Service Management Facility) with active respawn via a new `SYS_THREAD_NOTIFY (143)` syscall and a topological-cascade dependency graph; **Project Crossbow** virtual switch with O(1) VNIC provision/revoke, token-bucket rate limiting and locally-administered MAC generation; and **FMA** (Fault Management Architecture) with 6 prioritised correlation rules over disk-retry, ECC-correctable, ECC-uncorrectable, thermal and PCI-link events plus provenance-anomaly counters.
- A renewed **Wayland compositor** with Tokyo Night theme: layer-shell (`zwlr_layer_shell_v1`), xdg-shell popups + positioner, fractional scaling stub, wl_output v4, multi-shape cursor theme, anti-aliased `embedded-graphics` mono font, modern decorations with traffic-light buttons, vertical gradient wallpaper, per-rectangle damage tracking, animation framework with easing curves, and a feature-gated `tiny-skia` path-rendering backend.
- A **6-tier boot demo pipeline** that exercises real type-level code paths from the personality crates and gates on PASS markers. End-to-end verification reaches **16 PASS markers in a single boot** on commit `eb137e3`, with zero panics and zero triple faults.
- **Formal verification**: 6 TLA+ specifications (capabilities, IPC, scheduler, sot_capabilities, sot_provenance, sot_transactions) with matching TLC model-checking `.cfg` files. `just tlc-mc` runs all six and reports per-spec state counts.
- **Production hardening**: a strict CI gate (`cargo fmt -check` + `cargo clippy -D warnings`) on the kernel and core libraries, nightly ABI fuzzing of every public syscall with random

arguments, reproducible disk-image builds (`SOURCE_DATE_EPOCH`), a per-PR ABI-diff bot, automated GitHub Releases with Ed25519-signed SDK tarballs, and `cargo audit` + `cargo deny` on every push.

The proposition is straightforward: no open-source operating system in 2026 combines a verified microkernel core, a real NetBSD rump in-tree, capability-based deception with live migration, an Illumos-style service / network / fault management triad, and a formally-checked SOT exokernel layer. sotX is small enough (108 k LOC of Rust) to be auditable but rich enough to host real Linux binaries (`busybox`, `git clone`, dynamic glibc) and to drive Wayland clients with a modern theme.

2 Threat Model and Motivation

2.1 What sotX defends against

sotX targets two threat classes that conventional operating systems handle poorly:

Class A: post-exploitation lateral movement and persistence

Once an attacker gains code execution inside an unprivileged process on a Linux server, the standard playbook is well known: enumerate `/etc/shadow`, escalate via SUID binaries, install a persistence mechanism in `/etc/cron.d`, then pivot to neighbouring services through the kernel network stack. Conventional defences (SELinux, AppArmor, seccomp) are policy filters that the attacker either bypasses once or works around with documented techniques.

sotX changes the contract. Every kernel object is accessed through a capability, and *capabilities can be interposed* after the fact. The deception subsystem watches the per-CPU provenance ring, identifies the canonical `CredentialTheftAfterBackdoor` pattern (write a fake `/sbin/sshd`, then read `/etc/shadow`, then read `/proc/version`), and live-migrates the offending domain into a deception profile. The attacker is now talking to a fake `/proc/version` that reports Ubuntu 22.04, a tarpit `/etc/shadow`, and a fake OpenSSH banner — with no observable disconnect. Kill chains stall on the *wrong target*.

Project Crossbow extends this: as soon as a domain is moved into deception mode, the Crossbow virtual switch provisions a VNIC with a spoofed MAC and a 10 Mbps token-bucket rate limit. The attacker can no longer exfiltrate or pivot to a peer at line rate; they are syntactically isolated in a sandbox at O(1) provisioning cost.

Class B: silent storage / hardware degradation

Conventional storage stacks (ext4, XFS, NTFS) treat disk read retries as transient noise and surface them only after the underlying device fails. ECC-correctable RAM events are visible to mcelog but not to the storage layer. Thermal warnings and PCI-link degrades go to nobody.

The FMA module consumes the same provenance ring the deception subsystem uses, plus a hardware-fault history of 64 entries, and applies six prioritised correlation rules. **One uncorrectable ECC** fires `MigrateNow`. **Five correctable ECC events at the same address** fire `MigrateNow`. **Five disk-retry events at the same LBA within a 1 B-TSC window** fire `MigrateNow`. **Thermal warning AND >10 provenance-anomaly events** fires `MigrateNow`. The decision is fed into the Migration Orchestrator which evacuates Tier-1 / Tier-2 transactions off the failing region into a healthy frame *before* the silicon throws a fatal exception.

This is not magic. It is the same idea as Solaris FMA, applied to a microkernel with capability semantics where "move this transaction to a different memory region" is a finite, well-defined operation instead of the open-ended hand-wave it tends to become on Linux.

2.2 Why a microkernel for this

The deception and FMA layers cannot trust the very kernel they are trying to defend. On a monolithic kernel, an attacker who reaches `ring 0` via a single bug owns the deception detector itself. On sotX, the kernel implements *five primitives* and 35-ish syscalls, of which the deception machinery is entirely userspace (`services/init/` and the personality crates). A bug in the deception detector cannot escalate. A bug in the personality cannot escalate. The TCB is the verified microkernel plus the boot chain *and nothing else*.

The cost of this contract is two: (a) every cross-domain interaction goes through capability invocation rather than a direct function call, and (b) the userspace VMM, network stack, file system, and personality layers must be re-implemented or wrapped from upstream. We pay (a) with capability cache and fast IPC, and we pay (b) with the `rump-vfs` service (real NetBSD VFS, BSD filesystems and device drivers) plus the 60-function `rumpuser_sot.c` hypercall layer.

3 Architecture

3.1 The five-primitive microkernel

`kernel/src/` hosts the entire Ring 0 surface. The five primitives are:

1. **Scheduling** (`kernel/src/sched/`). Per-core ticket-locked run queues with work stealing, 4-level priority MLQ (`realtime/high/normal/low`), EDF for real-time threads, per-thread CPU-tick and memory-page resource limits enforced in the scheduler and frame allocator. The new `SYS_THREAD_NOTIFY` hook fires from `exit_current()` so SMF can detect dying services without polling.

2. **IPC** (`kernel/src/ipc/`). Synchronous endpoints (L4-style), asynchronous channels (lock-free SPSC ring buffers), notifications (one-shot event signals), routing layer for cross-node delivery. Capability transfer on `send` is supported. `call_timeout` (syscall 135) lets a caller wake up after N ticks if the callee never replies.

3. **Capabilities** (`kernel/src/cap/`). The capability table is a fixed-size array indexed by 32-bit handles with generation counters (ABA-safe). The Capability Derivation Tree (CDT) records delegation parent/child links so revocation cascades atomically. Every kernel object (thread, AS, endpoint, channel, frame, SO) is accessed through a capability with explicit rights bits.

4. **IRQ virtualisation**. The kernel exposes IRQs as capabilities. Userspace drivers register an `irq_cap` and receive notifications via the standard endpoint mechanism. The kernel handles only the bare minimum (LAPIC timer, IPIs, fault delivery).

5. **Frame allocation** (`kernel/src/mm/`). Bitmap physical-frame allocator (single-byte-per-frame metadata), slab allocator on top (for per-CPU caches), demand paging delegated to a userspace VMM via the global fault handler. Frame reference counting (`frame_refcount_inc`) is used by CoW fork.

The whole kernel fits in ∼18 k LOC of Rust including 1340 `unsafe` blocks (mostly in `arch/x86_64/`, all carrying `// SAFETY:` comments after PR #66). The boot binary is around 1 MiB.

3.2 The SOT exokernel layer

`kernel/src/sot/` adds 12 syscalls on top of the microkernel core. They implement Secure Objects, transactional updates, and capability interposition.

The kernel `ProvenanceEntry` is a `#[repr(C)]` 48-byte record containing `epoch`, `domain_id`, `operation`, `so_type`, `so_id`, `version`, `tx_id`, `timestamp`. Every `handle_tx_commit` and `handle_tx_abort`

#	Syscall	Purpose
300	`so_create`	Create a Secure Object (file/dir/process/socket/credential type)
301	`so_invoke`	Method call on an SO; routed through interposition if applicable
302	`so_grant`	Grant SO access to a domain (with optional interposition policy)
303	`so_revoke`	Revoke an SO grant (with epoch barrier)
304	`so_observe`	Observe an SO without acquiring rights
305	`sot_domain_create`	Create an isolation domain
306	`sot_domain_enter`	Switch into a domain
307	`sot_channel_create`	Create a typed SOT channel
308	`tx_begin`	Begin a transaction (`ReadOnly` / `SingleObject` / `MultiObject`)
309	`tx_commit`	Commit (single phase for tiers 0/1, second phase for tier 2)
310	`tx_abort`	Abort + rollback via WAL
311	`tx_prepare`	**Tier 5**: Tier 2 (MultiObject) prepare phase for two-phase commit
260	`provenance_drain`	Pop entries from the per-CPU provenance ring into a userspace buffer
261	`provenance_emit`	Inject a synthetic provenance entry (used by attacker simulator + FMA test)
262	`provenance_stats`	Read ring metadata (length, drops, total pushed)
143	**`thread_notify`**	**NEW (PR #51)**: register a notify-cap to be signalled when a target tid exits
144	`notify_poll`	Non-blocking poll for a pending notification

Table 1: The SOT and notify syscall surface, syscalls 143/144 added in the v0.2.0 Illumos Heirlooms wave for SMF active respawn.

emits a corresponding `OP_TX_COMMIT` / `OP_TX_ABORT` entry with `so_type = SOTYPE_TX_EVENT`, so userspace observers can drive ZFS / HAMMER2 snapshot managers from real kernel transaction events without polling.

3.3 KARL: per-boot ID randomisation

`kernel/src/karl.rs` derives a 64-bit `boot_seed` from `RDTSC` at boot time and exposes per-pool offsets. The effect is the same as OpenBSD's KARL kernel re-link for everything visible through the SOT syscall ABI:

```
Boot 1: boot_seed = 0x2895a574d42c28d7   tx_id starts at 3559653379
Boot 2: boot_seed = 0x726387273efe151f   tx_id starts at 1056833539
```

Listing 1: Per-boot KARL drift, observed across two cold boots.

Attackers cannot count from a known baseline. KARL is wired into the `Pool<T>` generation counters for tx, thread and capability pools.

4 The v0.2.0 Illumos Heirlooms Wave

The v0.2.0 batch (codename: **Project STYX**) takes inspiration from Illumos / OpenIndiana / Solaris and adds three major subsystems plus a Wayland compositor renewal and a 4-axis hygiene pass. All 20 units shipped as PRs #53–#72 over 2026-04-07/08 and merged to `sotX` HEAD `eb137e3`.

4.1 SMF (Service Management Facility)

`services/init/src/supervisor.rs` now hosts a 16-service fixed-size dependency graph. Each service entry carries:

```rust
pub struct SmfService {
    name: &'static [u8],
    deps: &'static [&'static [u8]],
    state: SmfState,            // Offline / Starting / Online / Failed / Maintenance
    restart_policy: RestartPolicy,  // Never / OnFailure / Always
    tid: Option<u64>,
    notify_cap: Option<u64>,
    respawn_count: u32,
}
```

When a registered service is spawned, `register_smf()` allocates a notification, calls `sys::thread_notify(thread, notify_cap)` so the kernel signals the notification on thread exit, and stores the notify-cap in the service slot. The `poll_and_respawn()` loop polls each notification non-blockingly and, on a death event, transitions the slot to `SmfState::Failed`, walks all dependents in topological order, and respawns each according to its restart policy. The *honeypot DB* → *deception services* cycle from the proposal is encoded as a concrete dependency edge in the test fixture.

Boot verification: a real `attacker` thread is spawned, the kernel reports its death, the supervisor detects the death event, respawns the attacker, and prints `=== SMF respawn: PASS ===`. The full cycle is observed end-to-end in QEMU on every boot.

The new kernel hook in `kernel/src/sched/mod.rs::exit_current()` fires `on_thread_exit()` which walks a fixed-size 64-entry `THREAD_NOTIFY` table under `spin::Mutex` and signals each matching notification. The double-check race pattern (`register_thread_notify` re-checks liveness after writing its slot, `on_thread_exit` races for the same flag) is documented and proven correct in the PR. The timer-ISR resource-limit kill path intentionally does *not* call `on_thread_exit` to avoid lock-order inversion (NOTIFICATIONS lock inside an interrupt handler); services with strict resource quotas should use `RestartPolicy::Always` so the next poll cycle catches them via passive liveness check.

4.2 Project Crossbow: virtual networking

`libs/sot-crossbow/` introduces a fixed-size 32-port `VirtualSwitch` with O(1) provision/revoke and a token-bucket rate limiter:

```rust
pub struct VirtualSwitch {
    ports: [Option<Vnic>; MAX_VSWITCH_PORTS],   // 32
    byte_counters: [u64; MAX_VSWITCH_PORTS],
    last_window_tsc: [u64; MAX_VSWITCH_PORTS],
}

impl VirtualSwitch {
    pub fn provision(&mut self, domain: DomainId, bw_kbps: u32) -> Result<u8, CrossbowError>;
    pub fn revoke(&mut self, port: u8);
    pub fn route(&mut self, src_port: u8, pkt_len: usize, now_tsc: u64) -> SwitchDecision;
    pub fn lookup_domain(&self, domain: DomainId) -> Option<u8>;
    pub fn vnic(&self, port: u8) -> Option<&Vnic>;
}
```

Each `Vnic` carries a locally-administered MAC derived from `(domain_id, tsc)` via a Knuth golden-ratio mix (`0x9E37_79B9_7F4A_7C15`), so adjacent domain IDs and consecutive provisionings always produce visibly different MACs. The first byte has the LAA bit set (`0x02`) and the multicast bit clear.

The token bucket refills once per 1-second TSC window; `bw_kbps * 125 = bytes/sec`. For a

10 Mbps cap (the default for sandboxed APTs) the bucket allows 1.25 MB/s. `route()` returns `SwitchDecision::Forward(port)`, `Drop`, or `RateLimited`.

The crate is wired into `personality/bsd/network/src/pf.rs`. When `PfInterposer::enable_deception(doma` flips a domain into deception mode, it also calls `crossbow.provision(domain, DEFAULT_BW_LIMIT_KBPS)`. `disable_deception(domain)` revokes the matching port. The result: an attacker that triggers the deception migration is *simultaneously* talking to a fake `/proc/version` *and* routed through a 10 Mbps spoofed-MAC VNIC. They can no longer pivot at line rate.

Test coverage: 18 unit tests for provision exhaustion, revoke + reuse, MAC uniqueness across all 32 ports, locally-administered bit, rate-limit trip, refill window, and TSC override. Boot-time smoke prints `=== Crossbow: PASS ===`.

4.3 FMA (Fault Management Architecture)

`libs/sot-fma/` introduces a small predictive engine with `FMA_FAULT_HISTORY = 64` hardware-fault entries and `FMA_PROVENANCE_HISTORY = 256` anomaly-counter entries:

```rust
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum HwFaultClass {
    DiskReadRetry, EccCorrectable, EccUncorrectable,
    ThermalWarn, PciLinkDegrade,
}

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum MigrationRecommendation {
    MigrateNow { reason: &'static str },
    MonitorClosely { reason: &'static str },
    NoAction,
}

pub struct FaultManagement { /* fixed-size, no_std, forbid(unsafe_code) */ }

impl FaultManagement {
    pub fn ingest_provenance(&mut self, entries: &[ProvenanceEntry]);
    pub fn ingest_hw_fault(&mut self, fault: HwFault);
    pub fn predict_failure(&self) -> MigrationRecommendation;
    pub fn fault_count(&self, class: HwFaultClass, window_tsc: u64, now_tsc: u64)
        -> usize;
}
```

The predictor (`libs/sot-fma/src/predictor.rs`) applies six prioritised rules:

1. Any single `EccUncorrectable` → `MigrateNow {reason: "ecc uncorrectable"}`
2. 5+ `EccCorrectable` at the same address → `MigrateNow {reason: "ecc soft errors clustering"}`
3. 5+ `DiskReadRetry` at the same location within a 1-billion-TSC window → `MigrateNow {reason: "disk read retries"}`
4. `ThermalWarn` AND `prov_window_anomalies > 10` → `MigrateNow {reason: "thermal + behavioral anomaly"}`
5. Any single fault → `MonitorClosely {reason: ...}`
6. Otherwise → `NoAction`

The 26 unit tests cover every threshold boundary, priority ordering, ring rotation, and TSC saturation. Boot-time smoke (`services/init/src/fma_demo.rs`) synthesises fake disk retries and uncorrectable ECC events and asserts the right recommendation, printing `=== FMA: PASS ===`.

Known follow-up: the local `ProvenanceEntry` mirror in `libs/sot-fma/src/lib.rs` is 40 bytes (rustc default field layout, no `#[repr(C)]`) while the kernel's struct is 48 bytes `#[repr(C)]`. Fine for the synthesised demo today; needs to be made layout-compatible before real drained-ring data is plumbed in.

5 The Wayland Compositor Renewal

`services/compositor/` went from a barebones engineering preview (8x8 bitmap font, 12x12 white arrow cursor, hardcoded BGRA `0xFF2D2D3D` background, no animations, no popups) to a modern Wayland compositor with the Tokyo Night palette in twelve parallel units (G1–G12).

5.1 Visual upgrade summary

5.2 Compositor architecture invariants

The whole compositor is `#![no_std]` and `#![no_main]`, single-threaded, with all per-client state in fixed-size arrays:

- `[WlClient; 8]` — max 8 concurrent clients
- `[Surface; 32]` — max 32 surfaces
- `[Toplevel; 16]` — max 16 toplevel windows
- `[ShmPool; 16]`, `[ShmBuffer; 32]`
- `[LayerSurface; 8]` for layer-shell
- `[Positioner; 16]`, `[Popup; 16]` for xdg-popup
- `[FractionalScale; 32]` for fractional scaling

The compose loop reads damage rects, clears those regions with the wallpaper, layouts layer surfaces, then draws in strict z-order: Background → Bottom → toplevels → Top → cursor → Overlay. Toplevel render area is reduced by the cumulative `exclusive_zone` of all visible layer surfaces so toplevels don't overlap a status bar.

6 Boot Verification: 16 PASS Markers

`just build && just run` on commit `eb137e3` reaches **16 PASS markers in a single boot** with zero panics:

```
=== compositor input: WIRED ===                    (line 218)
signify: verified=22 failed=0 missing=0            (line 237)
=== Signify boot chain: PASS ===                   (line 238)
=== STYX TEST: SOT exokernel syscalls 300-310 ===
STYX TEST: ALL PASS                                (line 279)
=== posix-test: PASS ===                           (line 446)
=== kernel-test: PASS ===                          (line 566)
RUMP-VFS-TEST: PASS                                (line 751)
=== Tier 3 demo: PASS ===                          (line 806)
=== FMA: PASS ===                                  (line 816)
=== Tier 4 demo: PASS ===                          (line 1409)
=== Crossbow: PASS ===                             (line 1417)
=== Tier 5 demo: PASS ===                          (line 16948)
=== abi-fuzz: 10000/10000 survived ===             (line 18702)
=== SMF respawn: PASS ===                          (line 19094)
DLTEST: PASS - dynamic linking works!              (line 19106)
DLTEST: mul PASS                                   (line 19108)
```

Listing 2: Live serial output (excerpt) from the post-v0.2.0 boot.

Each marker exercises real type-level code paths from the personality crates, not stubs. `abi-fuzz` alone hammers 10 000 random syscall arguments without panicking the kernel. `Tier 5` exercises a real two-phase commit, 5 000 random SOT syscalls with garbage args, concurrent transaction stress, and TCG-calibrated performance numbers next to published seL4 / NOVA / Fiasco / Linux references.

7 Production Hardening

7.1 Strict CI gate

The new `lint-strict-core` job in `.github/workflows/ci.yml` runs `cargo fmt -check` and `cargo clippy -target x86_64-unknown-none - -D warnings` on `sotos-kernel`, `sotos-common`, `sot-fma`, `sot-crossbow`, and `sotos-ld` *without* `continue-on-error`. Eleven inline clippy fixes (mostly `.flatten()`, `div_ceil`, `iter_mut`) plus crate-level targeted `#![allow(...)]` for known dead code with per-lint justification comments. *No* blanket `#![allow(warnings)]`.

A drive-by fix folds `.got` and `.got.plt` into `.data` in `kernel/linker.ld`, resolving a pre-existing `.text` / `.got` section overlap that was blocking `just build` on fresh checkouts.

7.2 Audit + supply chain

A new `audit.yml` workflow runs `cargo audit` (RustSec advisory database) and `cargo deny check` on every push and PR. `deny.toml` declares an explicit license allowlist (MIT, Apache-2.0, BSD-2/3, ISC, MPL-2.0, Unicode, Zlib, CC0, 0BSD), no GPL/AGPL, sources restricted to `crates.io`, and yanked crates denied. The job hard-fails on any RustSec advisory or license violation.

7.3 Reproducible builds

`repro.yml` runs a Linux + Windows matrix that builds `target/sotx.img` *twice on each host* via `python scripts/mkimage.py`, then compares the two SHA-256s. A final `compare` job downloads both host artifacts and asserts they're equal. Determinism is enforced by pinning all FAT32 directory-entry timestamps to `SOURCE_DATE_EPOCH` (default `1700000000`, 2023-11-14) via UTC `gmtime` in `mkimage.py`.

7.4 ABI fuzzing and ABI diff bot

`services/abi-fuzz/` is a no_std binary with a deterministic xorshift64 PRNG seeded by a fixed prime. It picks a random syscall from a curated whitelist (excluding the obviously-blocking and destructive numbers), fills six argument registers with random `u64`s, invokes via the raw `syscall6` helper, and loops for 10 000 iterations. The kernel must not panic, regardless of arg garbage. The harness prints `=== abi-fuzz: 10000/10000 survived ===` on success. `fuzz.yml` runs this nightly under QEMU.

`abi-diff.yml` parses the `Syscall` enum from `libs/sotos-common/src/lib.rs` on the PR base and head, computes the symmetric diff (added / removed / renumbered variants), and posts a sticky markdown comment on the PR. The script uses only Python stdlib so it runs on Linux and Windows. Currently the parser correctly identifies all 81 `Syscall` variants in the live tree.

7.5 Reproducible release pipeline

`release.yml` triggers on `git tag v*`, installs nightly Rust + `just` + QEMU, runs `just sigmanifest && just image`, asserts the 16 PASS markers in a 4-minute QEMU boot, runs `bash scripts/make-sdk.sh -version ${TAG}`, signs the tarball with Ed25519 (key sourced from `secrets.SIGNIFY_KEY_BYTES` as a base64-encoded 32-byte raw private key, decoded into a temp file with `trap` cleanup), and uploads `.tar.gz`, `.sha256` and `.sig` via `softprops/action-gh-release@v2`. A repo guard (`if: github.repository == 'sotomayorlucas/sotX'`) prevents fork-tag accidents.

8 Formal Verification

`formal/` contains six TLA+ specifications and matching `.cfg` files for TLC model checking. `just tlc-mc` runs all six and reports per-spec state counts:

```
==> TLC  ok  capabilities      (79137 distinct states, depth 10)
==> TLC  ok  ipc               ( 1498 distinct states, depth  5)
==> TLC  ok  scheduler         (  514 distinct states, depth 18)
==> TLC  ok  sot_capabilities  (  850 distinct states, depth  4)
==> TLC  ok  sot_provenance    ( 1462 distinct states, depth 13)
==> TLC  ok  sot_transactions  (  602 distinct states, depth  9)
tlc: 6 passed, 0 failed, 0 skipped
```

The first run of TLC found and fixed real bugs in five specifications: unbounded `CHOOSE` for NULL sentinels (anchored to one-element sets), an over-strict `Rollback` invariant (`sot_transactions::Rollback` required *every* WAL entry's `oldVal` to match `soValue`, but `RestoreFromWal` picks the *earliest* per object), stale WAL after abort, duplicate-thread `Enqueue` (`scheduler::Enqueue` allowed the same thread twice in a CPU's runqueue), and a looping `RECURSIVE Descendants` in `capabilities::Revoke` (replaced with an explicit fixed-point iteration bounded by `MaxCaps + 1`).

One open finding is parked: `sot_transactions::Atomicity` is violated when Tier 1 `T1Write` and Tier 2 `T2Begin` interleave on the same object; the spec's lock semantics don't honour the `T2` prepared state across tiers. The `.cfg` restricts checking to a single transaction so the structural invariants still verify.

`just tlc` runs the SANY syntax/level checker as a faster pre-check, and the `tla-tlc` CI job runs full model checking on every push.

9 Codebase Inventory

As of v0.2.0 (`eb137e3`, 2026-04-08):

The C inventory is fully justified: `limine/limine.c` is the external bootloader, `services/init/lkl/` and `services/lkl-server/` are the LKL hypercall bridge for the NetBSD `rump` layer (whose API is C and cannot be reasonably re-implemented in Rust without rewriting the whole rump kernel). All other userspace and kernel code is Rust. There is zero C++.

10 What sotX enables

The proposition for v0.2.0+ is short:

1. **Run real Linux binaries on a verified microkernel.** Static and dynamic glibc + musl, `busybox sh`, real `git clone`, real `nano` editor, all under the LUCAS personality with the kernel TCB capped at 18 k LOC.

2. **Detect post-exploitation lateral movement and live-migrate the attacker into a deception profile** without breaking their TCP state. Anomaly → Migration Orchestrator → Crossbow VNIC sandbox at 10 Mbps → fake `/proc/version`, fake `/sbin/sshd`, tarpit `/etc/shadow`.

3. **Predict hardware failures from soft signals** (disk retries, ECC corrections, thermal warnings) and evacuate Tier-1 / Tier-2 transactions *before* the silicon throws a fatal exception. The same provenance ring that feeds the deception subsystem feeds FMA.

4. **Respawn dependent services in topological order** when a critical one dies, via SMF. The kernel signals `SYS_THREAD_NOTIFY`; the supervisor cascades through the dependency graph; the attacker's TCP connection to a deception service survives the restart.

5. **Sign and verify the entire boot chain** with Ed25519, with pubkey-pinning to defeat swap-with-self-signed attacks. Tampering with one byte of any signed binary is caught at boot before the binary executes.

6. **Reproduce the disk image bit-for-bit** across Linux and Windows hosts (`SOURCE_DATE_EPOCH`), so a release SHA-256 actually means something.

7. **Drive a Wayland compositor with a modern Tokyo Night theme**, real anti-aliased fonts, multi-shape cursors, traffic-light decorations, layer-shell status bar, popups, animations, all in a `#![no_std]` userspace binary. The compositor talks to the kernel via the same capability-based IPC every other service uses.

8. **Verify the kernel's safety properties** with TLA+ model checking on six specifications, with the model checker running in CI on every push.

11 Roadmap and known follow-ups

11.1 Post-v0.2.0 follow-ups (non-blocking)

- **sot-statusbar happy path**: `init`'s `spawn_process` passes `cap=0`, so the statusbar can't allocate caps for the SHM pool. Needs kernel-side spawn (like the compositor) or cap-forwarding via `bootinfo_write`. The statusbar gracefully connects-and-waits today.

- **sot-fma `ProvenanceEntry` ABI compatibility**: local mirror is 40 bytes, kernel struct is 48 bytes `#[repr(C)]`. Fix: import the upstream type or add `#[repr(C)]` + a `const_assert!` on size.

- **Multi-monitor real implementation**: `wl_output` is currently a single-output stub. Hot-plug and multi-seat are deferred.

- **Real fractional scaling render path**: today the `wp_fractional_scale_v1` stub advertises 1.0× but the compositor renders at 1× regardless. HiDPI font rendering and surface scaling are deferred.

- **IME, clipboard, screen capture, gesture support, multi-seat**: real Wayland protocols, each a multi-day effort.

- **LTP (Linux Test Project) subset under LUCAS**: blocked on a host script to fetch and cross-build LTP.

- **WHPX/KVM IRQ model rework**: WHPX boots Limine but crashes on LAPIC injection (`Unexpected VP exit code 4`). The kernel's IRQ model isn't WHPX-compatible without a deeper rework. Today we use a TCG-calibration approach instead.
- **CHERI Morello target**: an `aarch64-morello-purecap` target plus an in-kernel CHERI enforcement path, so the software model in `services/sot-cheri` becomes a hardware-checked one.

11.2 Planned for v0.3.0

- Real OpenZFS `libzpool` link (today the snapshot manager is in-memory). Connect to SOT block device caps and replace the demo plumbing.
- `rump_init` curlwp rewrite to unblock the full `rump` kernel boot beyond the current ∼64 hypercall mark.
- bhyve VT-x / WHPX support so guest VMs can run with introspectable caps.
- HAMMER2 real userspace library + clustering for distributed deception.
- GPU rendering: DRM/KMS direct, OpenGL ES, or Vulkan. Today `tiny-skia` gives software path-based rendering as the immediate win.

12 Conclusion

sotX v0.2.0 ships a verified microkernel BSD personality OS in ∼108 k LOC of Rust, with a SOT exokernel layer, a real NetBSD `rump` kernel, a Linux ABI personality that runs real glibc and musl binaries, capability-based deception with live migration, SMF active-respawn supervision, Project Crossbow virtual networking, FMA predictive self-healing, a Tokyo Night Wayland compositor, and a production hardening pipeline that enforces formatting, lints, licenses, advisories, ABI surface, reproducible builds and boot-smoke testing on every PR. Six TLA+ specifications are model checked in CI on every push.

This is, to the author's knowledge, the only operating system in 2026 that combines all of these in a single auditable codebase. The current work was driven by an exploration of how Solaris, OpenIndiana, OpenBSD and Illumos solved problems that mainstream Linux distributions still hand-wave about, ported into a verified microkernel where the kernel TCB is small enough to be meaningfully audited and the userspace is modular enough that any single component can be replaced or upgraded independently.

The next release (v0.3.0) will focus on closing the four big follow-ups: real `libzpool`, `rump_init` curlwp, bhyve guest VMs, and HAMMER2 clustering for distributed deception.

Repository: `github.com/sotomayorlucas/sotX` *Branch:* `sotX` *HEAD:* `eb137e3` *Date:* 2026-04-08

Author: Lucas Sotomayor, BASE4 Security *Codename:* Project STYX

Was	**Is (after v0.2.0)**	**PR**
Solid #2D2D3D background	Tokyo Night vertical gradient (#1A1B26 → #10101C), 8.6 KiB row cache, optional BMP image tile path	#57
8×8 bitmap font, 95 ASCII glyphs	`embedded-graphics` `FONT_10X20` (large) + `FONT_6X10` (compact), anti-aliased, with a `FbTarget` adapter implementing `DrawTarget`	#56
12×12 white arrow cursor only	8 cursor shapes (Default, Pointer, Text, Wait, Move, ResizeNS, ResizeEW, NotAllowed) with hand-crafted BGRA glyphs and hot-spots, in static `rodata`	#55
24 px solid title bar, red square close button	28 px Tokyo Night title bar with three traffic-light circles (red #F7768E, yellow #E0AF68, green #73DACA, 14 px diameter), rounded corners	#62
Single `bool` damage flag, full-screen redraw	32-rectangle dirty list with coalesce-on-overflow and fullscreen-escalation if union > 3/4 of screen	#53
No popups (`xdg_wm_base` v2 advertised but only `xdg_toplevel` implemented)	Full `xdg_positioner` + `xdg_popup` with all anchor / gravity combinations and on-the-fly geometry recompute on	#70

Language	**File count**	**LOC**
Rust (.rs)	376+	∼108 k
C (.c, vendor + glue)	13	∼4.4 k
C++	0	0
Python (.py)	50+	∼5 k
TLA+ (.tla)	6	∼1.5 k
Markdown (.md)	16+	∼10 k
TOML (.toml)	105	—
Total	—	∼130 k
Crates in workspace		69
Kernel pools (static memory)		∼4.37 MiB
Largest single pool	`FRAME_REFCOUNT`	4 MiB
`unsafe` blocks (kernel)		1340
TLA+ specifications (TLC-checked)		6
PASS markers in a clean boot		16

Table 3: Codebase inventory after the v0.2.0 wave. C is justified (Limine bootloader glue + LKL/rump shims for the in-tree NetBSD `rump` layer); no migration needed.